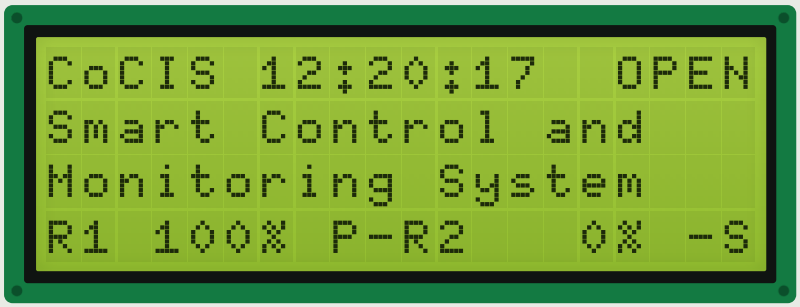


Group 15

Embedded Systems, 2026/2027.
College of Computing and Information Sciences, Makerere University.

Access control, occupancy records and automatic lighting for the CoCIS main entrance and two lecture rooms.

1 **Mujuni Innocent**
Overview, integration, storage and synchronisation

2 **Martha Namuyimbwa**
Entrance access control

3 **Ochen Felix**
Lecture-room lighting

4 **Nakimuli Jollyne Flavia**
Management console and admin login

5 **Tamara Alice Nazziwa**
Testing and cloud dashboard

Mujuni (10 s). Good morning. We are Group 15, and this is our Smart Control and Monitoring System for CoCIS.

I am Mujuni Innocent, registration number 24/U/07155/PS. Each of us will present the part we built, in the order on the screen.

Two problems, one pilot

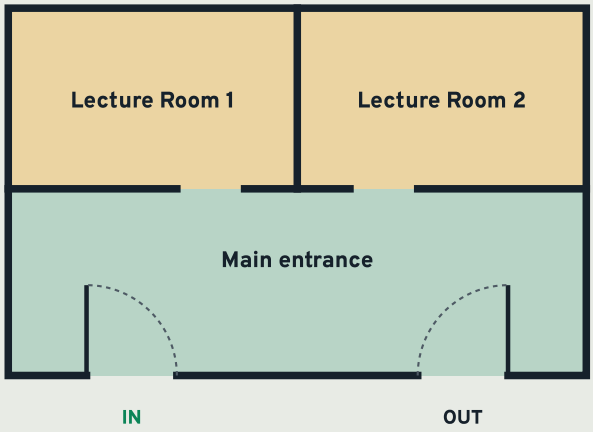
Valuable equipment

The college needs to control who comes in, who leaves, and who is still inside.

High electricity bills

Lights stay on in lecture rooms that nobody is using.

Management asked for a pilot on the main entrance and two lecture rooms, run by ESP32 terminals.



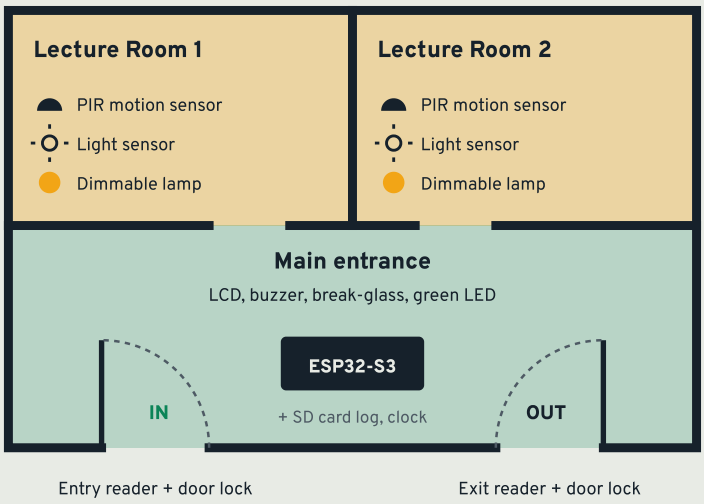
Mujuni Innocent 24/U/07155/PS
Overview, integration, storage and synchronisation

2 / 17

Mujuni (20 s). CoCIS has two problems: valuable equipment, so it must control who comes and goes and know who is inside; and high electricity bills, partly from lights left on in empty lecture rooms.

Management asked for a pilot on the main entrance, with separate entry and exit doors, and two lecture rooms.

Three terminals on one ESP32-S3



Main entrance hub
Two RFID readers and two servo door locks check people at entry and exit, keep the list of who is inside, and enforce opening hours.

Lecture Rooms 1 and 2
Motion and light sensors switch and dim each lamp, use daylight, and report the energy saved.

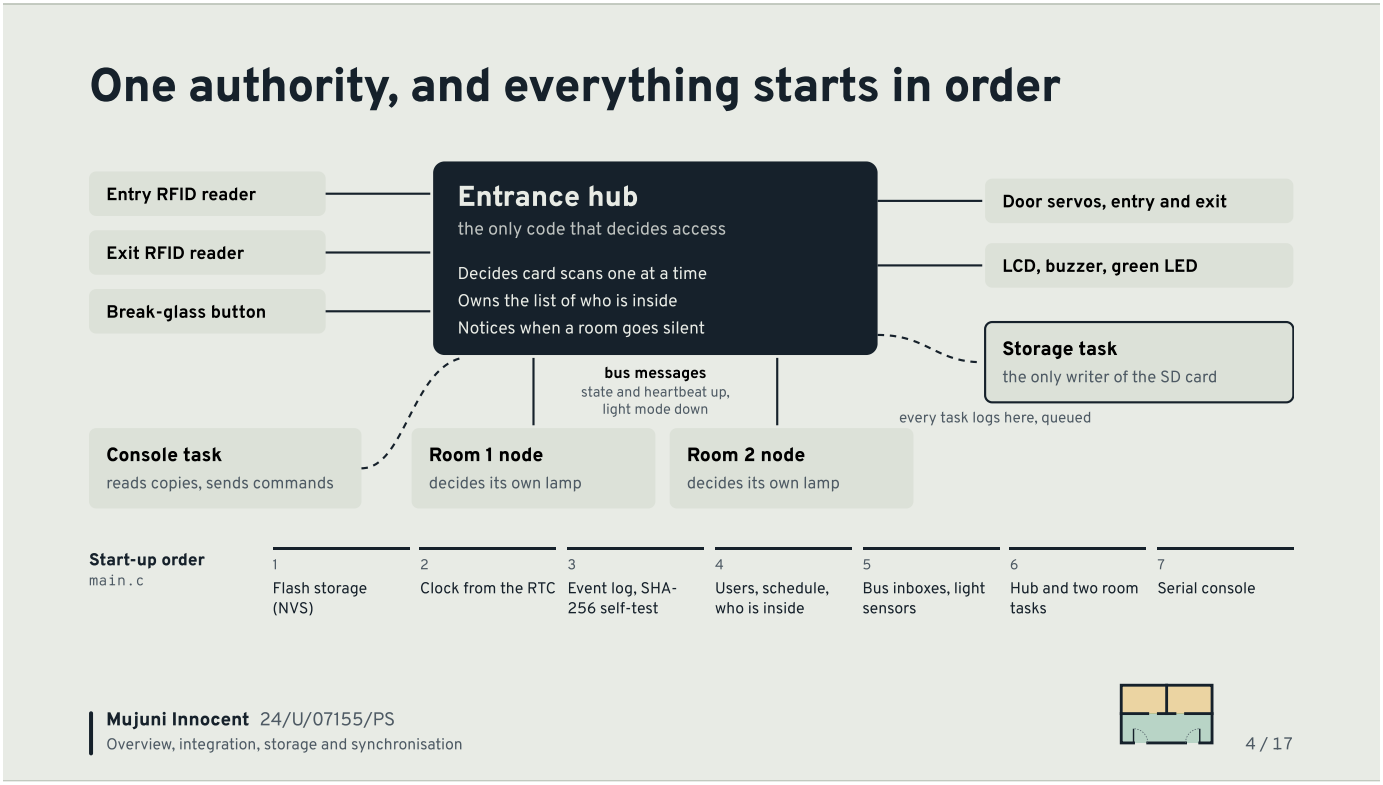
Runs offline
No network needed. Built with ESP-IDF and FreeRTOS, simulated in Wokwi.

Mujuni Innocent 24/U/07155/PS
Overview, integration, storage and synchronisation
3 / 17

Mujuni (35 s). Our system is built on the ESP32-S3 with ESP-IDF and FreeRTOS, and it has three terminals.

First, the main entrance hub. It uses two RFID readers and servo door locks to verify users at entry and exit, track who is inside, and enforce opening hours, anti-passback and an emergency break-glass override.

Second and third, lecture rooms 1 and 2. They use motion and light sensors to switch and dim their lamps automatically, use daylight, and report the energy saved to the hub.



Mujuni (30 s). My contribution. My personal role was system integration, resource synchronisation, NVS persistence and SD card storage.

First, in main.c, I designed the boot sequence along the bottom: time, storage and shared state start in the right order before the FreeRTOS tasks of the three terminals.

The hub is the single authority: the readers, the button and the rooms only send it messages.

Every shared thing has one owner

Who is inside, and every access decision	Only the hub task writes it. Scans wait in its inbox and are decided one at a time.
The SD card	Only the storage task touches it. Every other task queues its log lines.
Users, schedule, login state	One mutex each, held for a quick copy. Readers get a copy, never a pointer.
Settings and the ledger in flash	Saved to NVS with an atomic commit, so the list of people inside survives a power cut.
Messages on the bus	A sequence number on every message. The receiver drops repeats and replays.

```
bool bus_send(src, dst, msg, wait)
{
    xSemaphoreTake(s_send_lock[src], ...);
    msg→src = src;
    msg→seq = s_next_seq[src] + 1;
    bool sent = xQueueSend(s_inbox[dst],
                           msg, wait);
    if (sent)
        s_next_seq[src] = msg→seq;
    xSemaphoreGive(s_send_lock[src]);
    return sent;
}
```

bus.c, simplified. The per-sender lock makes "take the next number, queue the message" one step. We found that race in code review and fixed it.

Mujuni Innocent 24/U/07155/PS
Overview, integration, storage and synchronisation



5 / 17

Mujuni (25 s). Second, in bus.c, I built the message bus on FreeRTOS queues. Each message carries its sender and a sequence number, and a per-sender mutex keeps the numbers in order, so the receiver can safely drop duplicates and replays.

That is our answer to requirement 2.7: every shared resource has exactly one owner or one mutex.

Third, in occupancy.c and nvs_store.c, the list of people inside is protected by a mutex and saved to flash, so it survives power cuts.

The log notices if anyone edits it

```
000008|2026-10-28T11:35:49|ACCESS|uid=01020304 name=Alice side=IN ok=1 reason=OK |e9761b4b…
000009|2026-10-28T11:35:51|ACCESS|uid=01020304 name=Alice side=IN ~~ok=0 reason=ANTI_PASSBACK~~ ok=1 reason=OK |e9037d46…
000010|2026-10-28T11:35:53|ACCESS|uid=01020304 name=Alice side=OUT ok=1 reason=OK |18953a92…
000011|2026-10-28T11:35:54|ACCESS|uid=C0FFEE99 name=unknown side=IN ok=0 reason=UNKNOWN_CARD |3e562ebe…
```

```
hash = SHA-256( previous hash | line )
```

One file per day on the SD card. Every line is saved with `fsync`. With no card, the log is kept in RAM.

```
cocis> log verify
TAMPERED: hash chain broken at line 9 (of 9 checked).
```

Mujuni Innocent 24/U/07155/PS
Overview, integration, storage and synchronisation



6 / 17

Mujuni (25 s). Finally, in storage.c, I built a gatekeeper task: the only code that writes the SD card. All events go into daily log files.

Each line's hash includes the previous line's hash, so the lines form a chain. log verify recomputes it.

[Press] If someone changes Alice's refused scan to "granted", the hash no longer matches, and log verify points to line 9.

Thank you. I now hand over to Martha.

If asked: without a secret key, someone who rewrites every hash could hide an edit, and cutting off the last lines is not detected. An off-device copy (the dashboard) covers that.

One tap at the entry reader



Alice
Student
01020304

The Blue card in the Wokwi reader's panel. Tap it and the LCD shows this.

1. The reader task reads the card's UID with our MFRC522 driver and sends `CARD_SCAN` to the hub.
2. The hub looks the card up: Alice, student, enabled.
3. It gathers the facts: is she inside, is the building open, is there a lockdown?
4. `access_decide()` returns the verdict: `OK`
5. She is marked inside, and only the entry door opens, for 3 seconds.
6. An `ACCESS` line goes to the log; the LCD and buzzer tell her.

Martha Namuyimbwa 24/U/09436/PS
Entrance access control



7 / 17

Martha (40 s). I am Martha Namuyimbwa, registration number 24/U/09436/PS. I built the entrance access control.

When Alice taps her blue card at the entry reader: **[press]** the reader task reads the card with my MFRC522 driver and sends a card-scan message to the hub. **[press]** The hub looks her up. **[press]** It gathers the facts. **[press]** access_decide gives the verdict.

[press] If granted, she is marked inside and only the entry door opens, for three seconds. **[press]** Every scan, granted or refused, is logged, and the LCD and buzzer give feedback.

The rules are one function: the first match wins

```
if (!req→known)     return ACCESS_UNKNOWN_CARD;

if (!req→enabled)   return ACCESS_DISABLED;

if (entering while inside || leaving while outside)

                    return ACCESS_ANTI_PASSBACK;

if (req→lockdown)   return privileged ? OK_PRIVILEGED : LOCKDOWN;

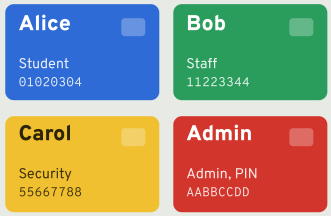
if (req→schedule_open)   return ACCESS_OK;

if (privileged)     return ACCESS_OK_PRIVILEGED;

if (leaving && req→escort_active)   return ACCESS_OK_ESCORT;

return ACCESS_OUTSIDE_HOURS;
```

Anti-passback means one card can't let two people in: no entering twice without leaving.



Security and admin may pass at any hour. No hardware calls, so it is unit tested on a PC.

Martha Namuyimbwa 24/U/09436/PS
Entrance access control



8 / 17

Martha (40 s). The rules are one pure C function in access_policy.c, read top to bottom; the first rule that matches wins.

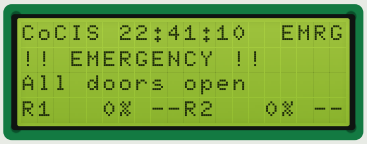
Unknown or disabled cards are refused. Anti-passback: you cannot enter twice without leaving, so one card cannot let two people in, and the list of who is inside stays true. Lockdown lets only security and admin through. Inside opening hours everyone passes. After hours only privileged roles pass, or ordinary users leaving during an escort window.

These are the four demo cards. Because the function has no hardware calls, we unit test every rule on a PC.

After hours: held inside, never trapped

- **Bob tries to leave after hours: refused.**
  `ACCESS name=Bob role=STAFF side=OUT ok=0 reason=OUTSIDE_HOURS`
- **Security is told someone is waiting.**
  `ALERT code=AFTER_HOURS_EXIT detail=Bob`
- **Carol from security leaves: a 60 s escort window opens.**
  `ACCESS name=Carol role=SECURITY side=OUT ok=1 reason=OK_PRIVILEGED`
- **Bob leaves with her, and the exit is recorded.**
  `ACCESS name=Bob role=STAFF side=OUT ok=1 reason=OK_ESCORT`

Martha's code: `drivers/mfrc522.c` `nodes/rfid.c` `core_logic/access_policy.c` `nodes/hub_access.c` `nodes/door.c`



Break-glass overrides every rule
Both doors are held open and an alert is logged. Only an admin can clear it.

Martha Namuyimbwa 24/U/09436/PS
Entrance access control



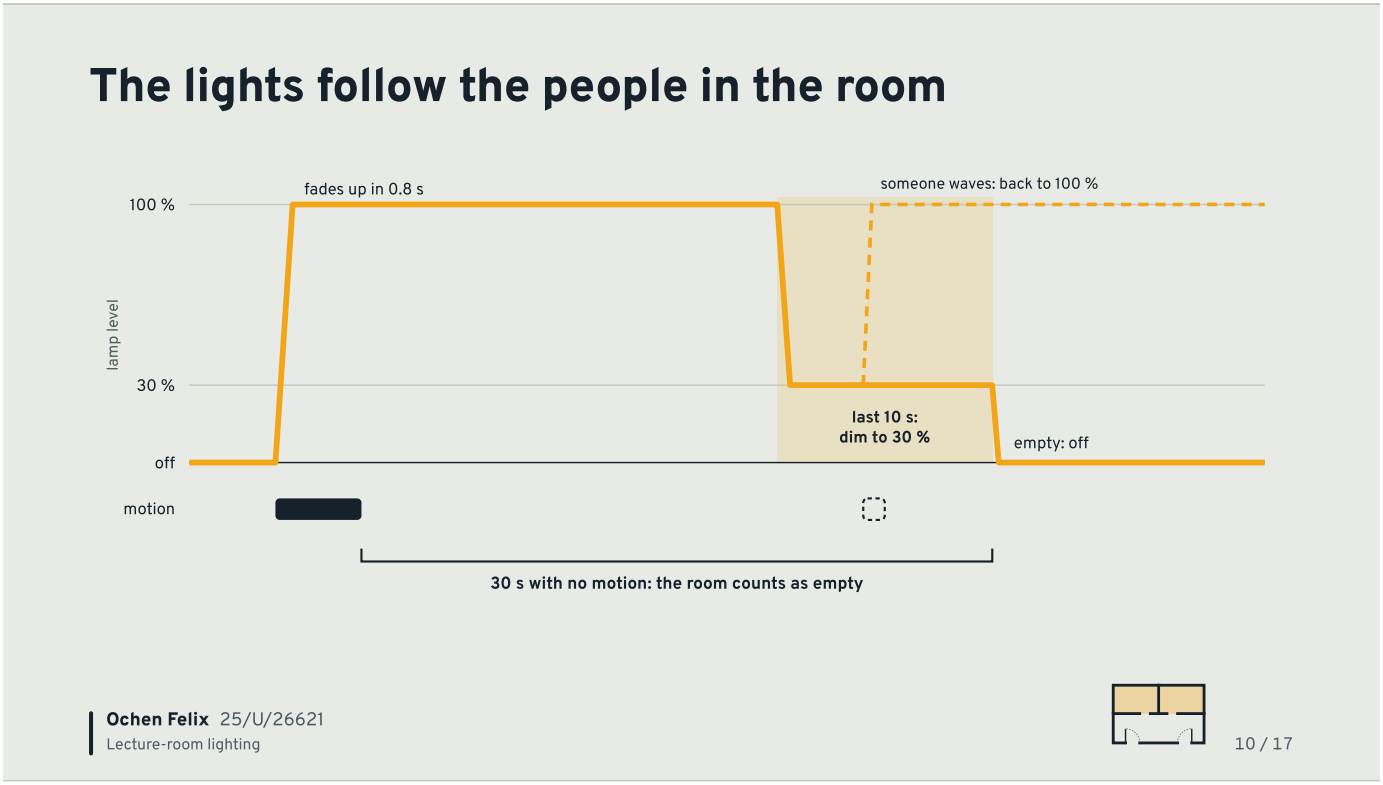
9 / 17

Martha (40 s). The brief says people still inside may not leave after hours. We enforce that, but we made sure nobody is trapped.

Bob is refused at the exit, and an alert tells security someone is waiting. When Carol from security leaves, a 60-second escort window opens, and Bob can leave with her. His exit is still recorded.

In an emergency, the break-glass button holds both doors open and logs an alert; only an admin can clear it.

My files are on the screen: the RFID driver and reader task, the access rules, the scan flow and the doors. Over to Ochen.

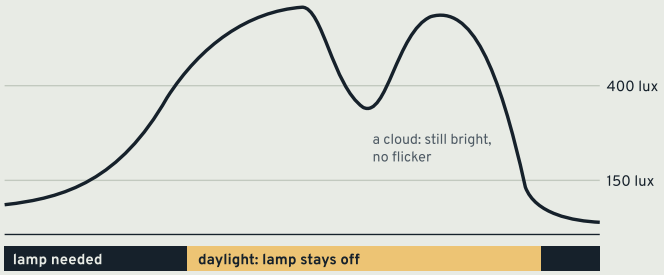


Ochen (45 s). I am Ochen Felix, registration number 25/U/26621. I built the lecture-room lighting.

Each room has its own task that runs five times a second and decides its own lamp, so it keeps working even if the hub goes silent.

Motion makes the room occupied and the lamp fades up. A PIR only sees movement, so we wait: with no motion for 30 seconds the room counts as empty. For the last 10 seconds the lamp dims to 30 percent as a warning. **[press]** If someone is still there, they move, and it goes straight back to 100 percent.

Daylight harvesting, and the energy it saves




```
if (lux >= cfg→bright_lux)      r→daylight = true;
else if (lux <= cfg→dark_lux)   r→daylight = false;
/* in between: keep the last decision */
```

saved = baseline − used

Baseline: a 360 W lamp bank left on for all opening hours. Used: the real lamp level over time.

```
cocis> rooms
Room Link   Occupancy Light  Lux  Saved Wh
1    online vacant       0%  650       0.4
2    online vacant       0%  100       0.0
```

An ENERGY line per room every 10 minutes.

Ochen's code: `room_logic.c` `room_node.c` `light_sensor.c` `actuators.c`

Ochen Felix 25/U/26621
Lecture-room lighting



11 / 17

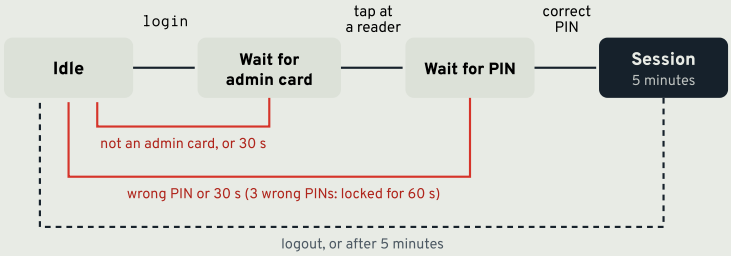
Ochen (45 s). Daylight harvesting: at 400 lux or more, the lamp stays off even in an occupied room. It only comes back when the light drops to 150 lux.

The gap between the two thresholds is hysteresis, so a passing cloud does not make the lamp flicker. That is these three lines of code.

For energy, every step adds what the lamp really used. During opening hours it also adds a baseline: a 360-watt lamp bank left on all day. The difference is the energy saved, logged every 10 minutes for each room.

My files are on the screen. Over to Nakimuli.

Changing the schedule takes a card and a PIN




```
cocis> schedule set 07:00 18:00 weekdays
Admin session required.
cocis> login
Tap the ADMIN card (Red card) at
either RFID reader within 30 s,
then type 'login <PIN>' here.
[LOG] AUTH uid=AABBCCDD result=CARD_OK
cocis> login 1234
Logged in: admin session started.
cocis> schedule set 07:00 18:00 weekdays
Schedule updated.
[LOG] CONFIG key=schedule
      value=07:00-18:00_-MTWTF-
      version=2 by=AABBCCDD
```

Something you have
The admin card, at either reader.

Something you know
A 4-digit PIN, stored only as a salted SHA-256 hash.

Everything is logged
Each step is an AUTH line; a lockout raises an alert.

Nakimuli Jollyne Flavia 24/U/0953
Management console and admin login



12 / 17

Nakimuli (50 s). I am Nakimuli Jollyne Flavia, registration number 24/U/0953. I built the management interfaces and the authentication.

The brief says only authorised staff may change the access schedule. So every change needs two factors: something you have, the admin's card tapped at a reader, and something you know, a 4-digit PIN typed on the console.

On the right, the schedule change is refused, then accepted after login. Three wrong PINs lock login for 60 seconds, and a session ends after 5 minutes. The PIN is never stored, only a salted SHA-256 hash. Every step is logged.

If asked about tap: it is a Wokwi test aid that injects a card from the console; on real hardware it would be removed or require an admin session.

The serial menu answers management's questions

Who is in the building right now?
`who`

Are the rooms lit, and what did we save?
`rooms`

Is the building open? Is anything wrong?
`status`

Has anyone edited the records?
`log verify`

Change the hours (admin only)
`schedule set 07:00 18:00 weekdays`

Nakimuli's code: `cli/` `core_logic/auth_fsm.c` `services/auth.c` `ui/ui.c`

```
cocis> status
Time         : 2026-10-28 11:42:46 Wed (EAT)
Schedule     : 07:00-22:00_-MTWTFS (v3)
               OPEN, closes in 618 min
Lockdown     : off   Emergency: off
Inside       : 3 people
Access       : 3 granted, 0 denied, 2 alerts
Storage      : SD card OK, last seq 15
Bus          : 0 duplicate frames rejected
RFID readers: entry OK, exit OK
Admin        : logged in as AABBCCDD
Room 1       : online, vacant, light   0%, 650 lux
Room 2       : online, vacant, light   0%, 100 lux
```

Nakimuli Jollyne Flavia 24/U/0953
Management console and admin login



13 / 17

Nakimuli (50 s). The serial menu is built on ESP-IDF's console. It is designed around management's questions.

"who" prints the muster list with entry times. "rooms" shows lights and energy. "status", on the right, is the whole system on one screen. "log verify" checks the records.

Anyone can read; only an admin session can change things. I also wrote the LCD and buzzer feedback in ui.c. My files are on the screen. Over to Tamara.

Tested before we demoed it

On a PC

```
$ make test
schedule  ok
access    ok
room      ok
auth      ok
types     ok

70 checks, 0 failures
```

The rules are pure C, so gcc tests them with `-Wall -Wextra -Werror`.

In Wokwi

Three scripts run the real firmware: they tap cards, move the light sliders, type commands, and fail unless the expected log lines appear.

`access.test.yaml` pass

`demo.test.yaml` pass

`rules.test.yaml` pass

Firmware build: 0 warnings.

Bugs we caught and fixed

A crash loop when a lamp faded on (hardware fade interrupt): now faded in software.

Two senders could reorder bus messages: per-sender lock.

A PIN typed after the timeout was accepted: expiry checked first.

LCD rows overflowing: every row is exactly 20 characters.

Tamara Alice Nazziwa 25/U/03596/PS
Testing and cloud dashboard



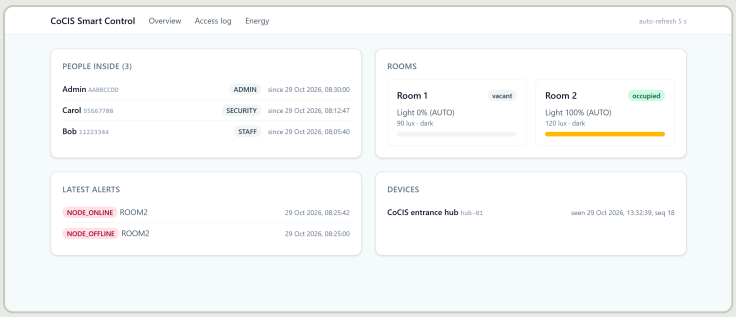
Tamara (55 s). I am Tamara Alice Nazziwa, registration number 25/U/03596/PS. I did the testing and the cloud dashboard.

The decision logic has no hardware calls, so it compiles and runs on a PC with all warnings treated as errors: 70 checks, zero failures.

Three Wokwi scenario scripts drive the real firmware in the simulator: they present cards, move the light sliders and type commands, and fail unless the expected log lines appear. All three pass.

Testing found real bugs, which we fixed. The worst was a crash loop: the hardware lamp-fade interrupt kept firing, so we fade the lamp in software instead.

An optional dashboard that only trusts signed data



Tamara's code: `test/host/` `test/wokwi/` `dashboard/`

Signed
Every batch carries an HMAC-SHA256 signature. A bad signature or a stale timestamp gets HTTP 401.

Stored once
The same 18 events sent twice are stored once: the second run reports 18 duplicates.

Optional
Next.js and Postgres on Vercel. The building never depends on it; the hub's uploader is the next step.

Tamara (35 s). The dashboard is an optional extra built with Next.js and a Postgres database, deployed on Vercel. It shows who is inside, the rooms, alerts and the access log.

It only accepts event batches signed with HMAC-SHA256 and rejects stale timestamps. Each event is stored once, so sending a batch again changes nothing.

The building never depends on it. Today it is fed by a signed sample sender; connecting the hub's uploader is the next step.

What the brief asked for, and what we added

The brief

2.1	RFID entry and exit, doors, who is inside
2.2	Lights that follow occupancy
2.3	Three coordinated terminals
2.4	Records on an SD card
2.5	Serial menus for management
2.6	A schedule only authorised staff can change
2.7	Synchronisation without race conditions

Added

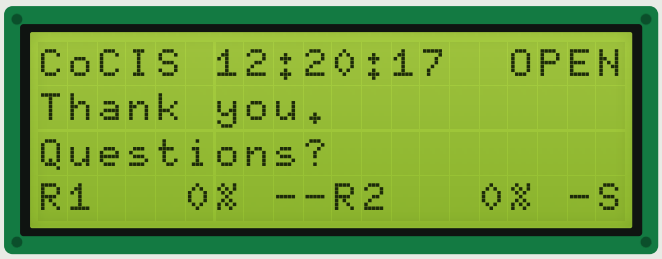
- Tamper-evident log
- Two-factor admin login
- Anti-passback and lockdown
- Escort window and after-hours alert
- Break-glass override
- Daylight harvesting
- Dim warning before lights off
- Energy used and saved
- Room heartbeats and offline alerts
- Keeps running without SD card or clock
- Signed cloud dashboard

Tamara Alice Nazziwa 25/U/03596/PS
Testing and cloud dashboard



16 / 17

Tamara (15 s). To sum up: every requirement in the brief, on the left, is met. On the right are the features we added to make the system more secure, efficient, reliable and useful to management.



Run it in your browser
wokwi.com/projects/479125502548914177

Video demonstration
youtu.be/BQb2dwnprf4

Scan to run the simulation
Press play in Wokwi, then tap a card

Tamara, or Mujuni (10 s). Thank you for listening. The Wokwi link runs the whole system in the browser, and the video shows each of our parts. We are happy to take questions.